# COMPUTER ARCHITECTURE

VNU - UNIVERSITY *of* ENGINEERING *&* TECHNOLOGY

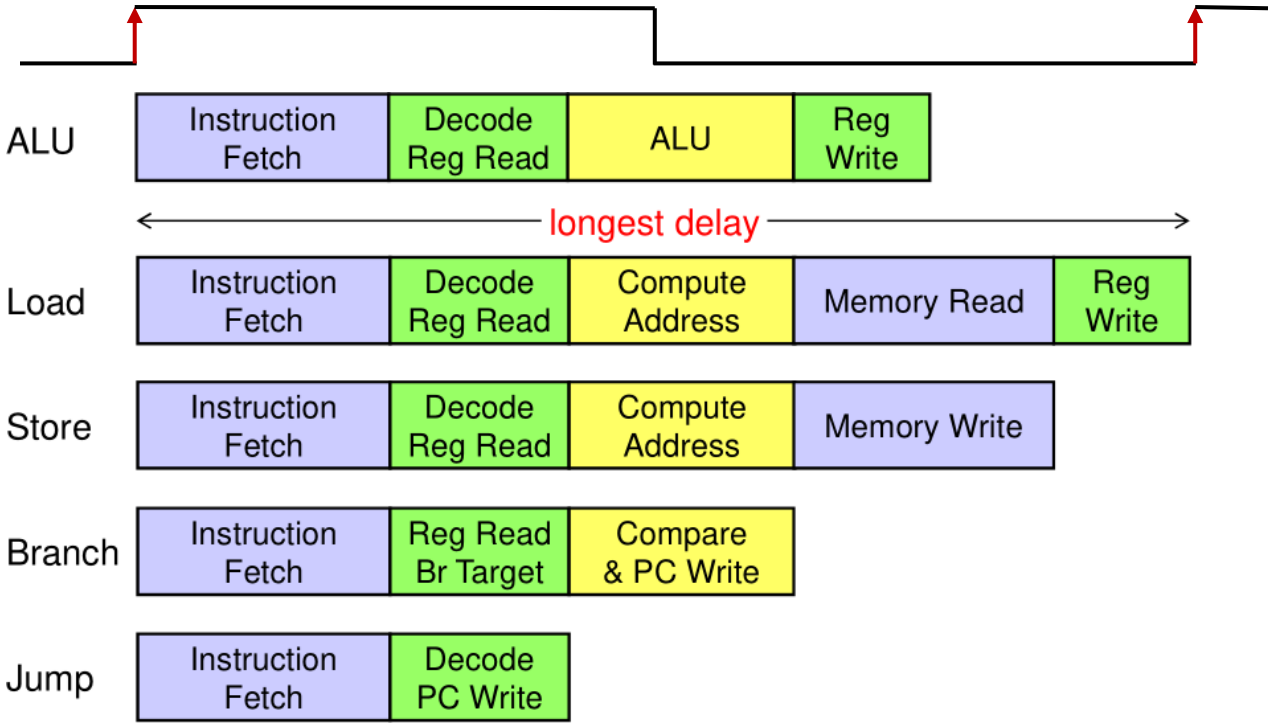
# LECTURE 6: Pipelined Processor

CONTENTS

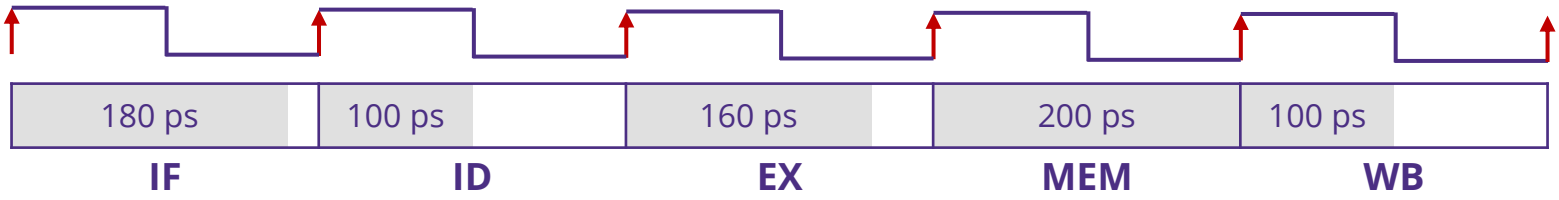
- Single-cycle vs. multicycle processors
  - Concepts
  - Drawbacks
- Pipelined processors
  - Introduction
  - Building the datapath & control unit
- Pipeline hazards
  - Structural hazards
  - Data hazards & forwarding

# Drawbacks of Single Cycle Processor

- All instructions take as much time as the **slowest** instruction
  - Not all instructions need all 5 stages



# Multicycle Processor




> Reduce cycle time to accommodate one stage per clock cycle
  - Clock cycle time is now constrained by longest stage
  - Simpler instructions take fewer cycles

| Instruction | # cycles | Instruction | # cycles |
|---|---|---|---|
| ALU & store | 4 | Branch | 3 |
| Load | 5 | Jump | 2 |

> Comparison with single-cycle processor
  - ✓ Less waste → higher overall performance
  - ✗ Design and implementation is more complicated

# Multicycle Processor: Performance Analysis

- Assume the following operation times

| Instruction Class | Instruction Memory | Register Read | ALU Operation | Data Memory | Register write | Total time |
|---|---|---|---|---|---|---|
| ALU | 200ps | 150 ps | 180ps | | 150 ps | 680ps |
| Load | 200ps | 150 ps | 180ps | 200ps | 150 ps | 880ps |
| Store | 200ps | 150 ps | 180ps | 200ps | | 730ps |
| Branch | 200ps | 150 ps | 180ps ← | Compare & write PC | | 530ps |
| beq | 200ps | 150 ps ← | Decode & write PC | | | 350ps |

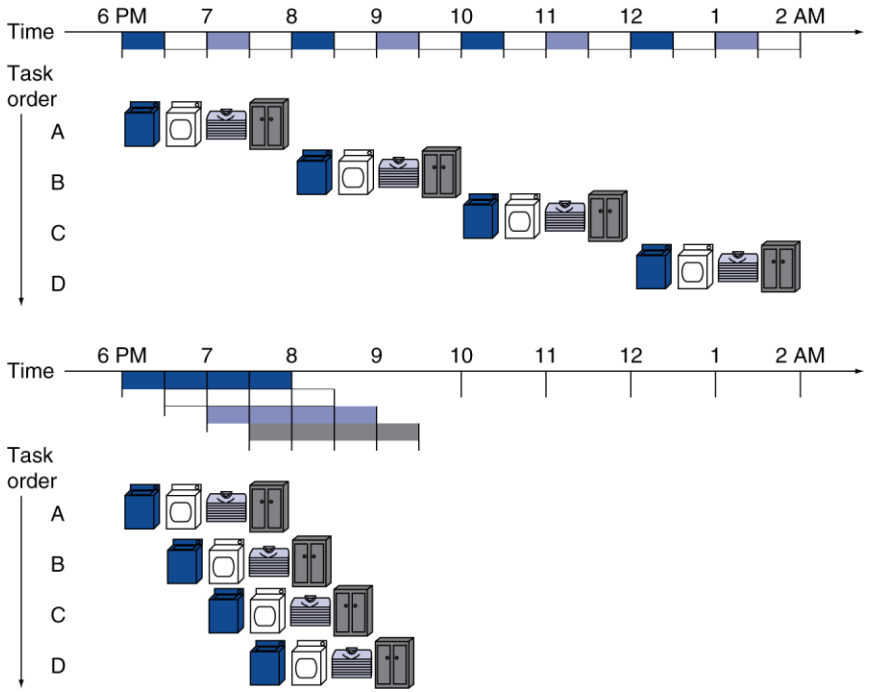
  - Instruction mix: 40% ALU, 20% Load, 10% store, 20% branch, & 10% jump

- Clock cycle time
  - Single cycle = max(ALU, load, store, branch, jump) = 880 ps
  - Multicycle = max (200, 150, 180) = 200 ps

- Speedup
  - Multicycle CPI = 0.4×4 + 0.2×5 + 0.1×4+ 0.2×3 + 0.1×2 = 3.8 cycles.
  - Speedup = 880 ps / (3.8 × 200 ps) = 880 / 760 = 1.16 times.

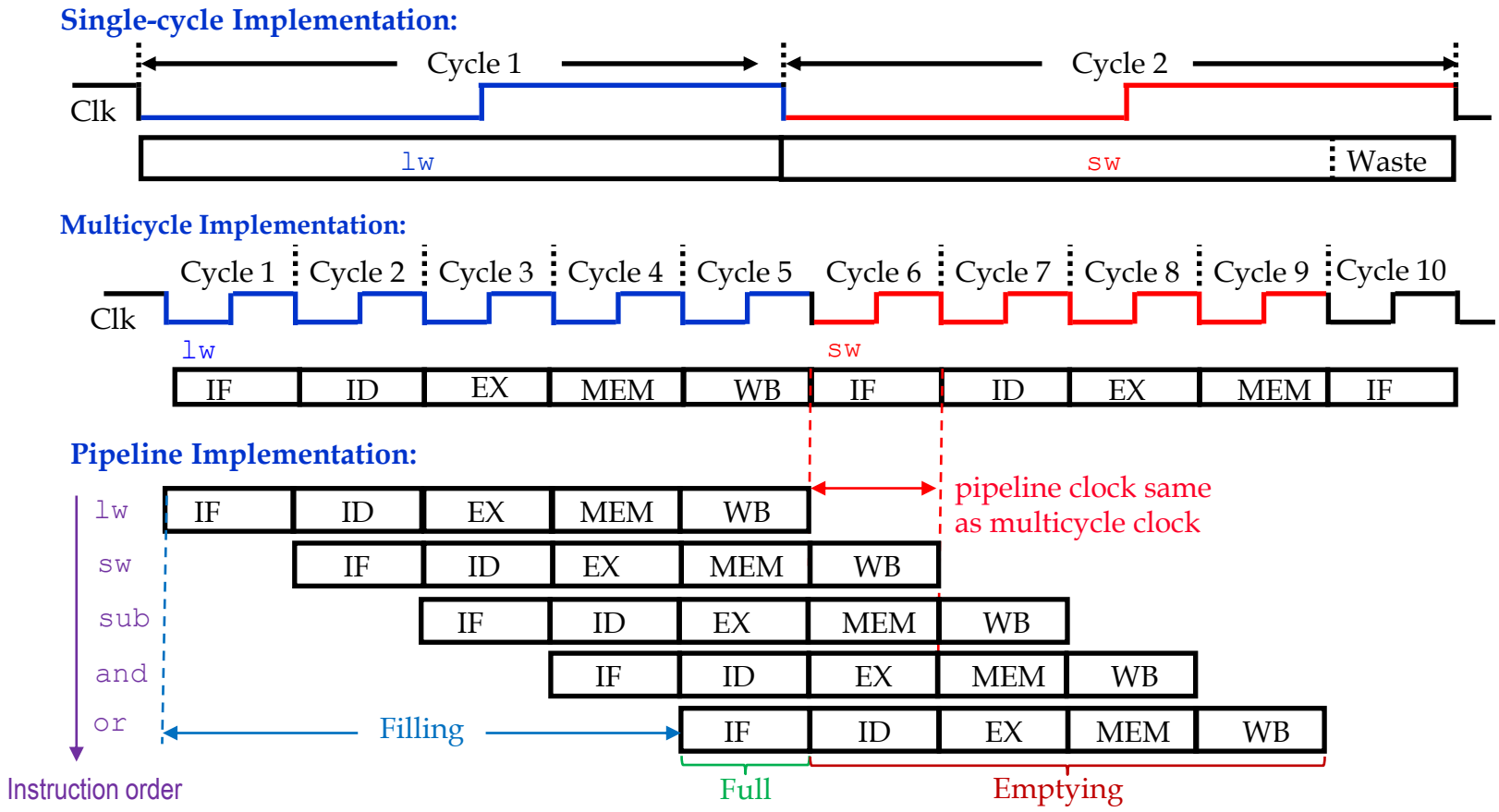
# Pipelining Idea

- Limitations of the multicycle design:
  - Some HW resources are **idle** during different phases of the instruction cycle.
  - **Most** of the datapath is idle when a memory access is happening.
- Pipelining idea in laundry
  - 4 loads: speedup $= \frac{8}{3.5} = 2.3$
  - $n$ loads: speedup $= \frac{2n}{0.5n+1.5}$
  - $n \to \infty$: speedup = #**stages**
- Ideal pipeline
  - Identical operations
  - Independent operations
  - Uniformly partitionable Suboperations
  - For non-ideal pipelines, speedup is **less**.



# Summary: Single-cycle vs. Multi-cycle vs. Pipelined

> Assuming 5-state pipeline



# Pipelined Processor: Performance Analysis

> Find the speedup of pipelined vs. single-cycle implementation

| Instruction Class | Instruction Memory | Register Read | ALU Operation | Data Memory | Register write | Total time |
|---|---|---|---|---|---|---|
| ALU | 200ps | 150 ps | 180ps | | 150 ps | 680ps |
| Load | 200ps | 150 ps | 180ps | 200ps | 150 ps | 880ps |
| Store | 200ps | 150 ps | 180ps | 200ps | | 730ps |
| Branch | 200ps | 150 ps | 180ps ← Compare & write PC | | | 530ps |
| beq | 200ps | 150 ps ← Decode & write PC | | | | 350ps |

> Pipelining clock cycle time = max (200, 150, 180) = 200 ps

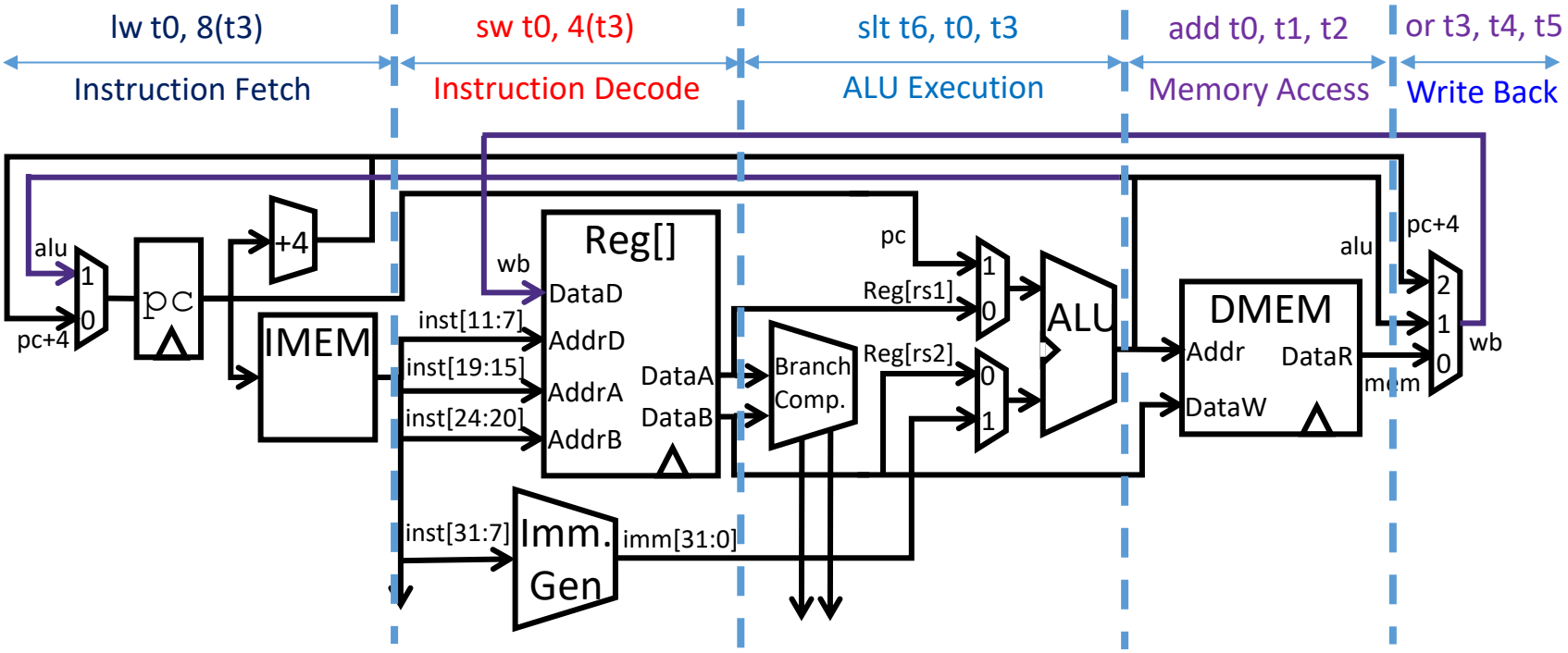
> Execution time for the first 5 instructions:

- Single cycle = 880×5 = 4400 ps
- Pipelining = 200×5+800 = 1800 ps

} Speedup = 4400/1800 ≈ 2.44

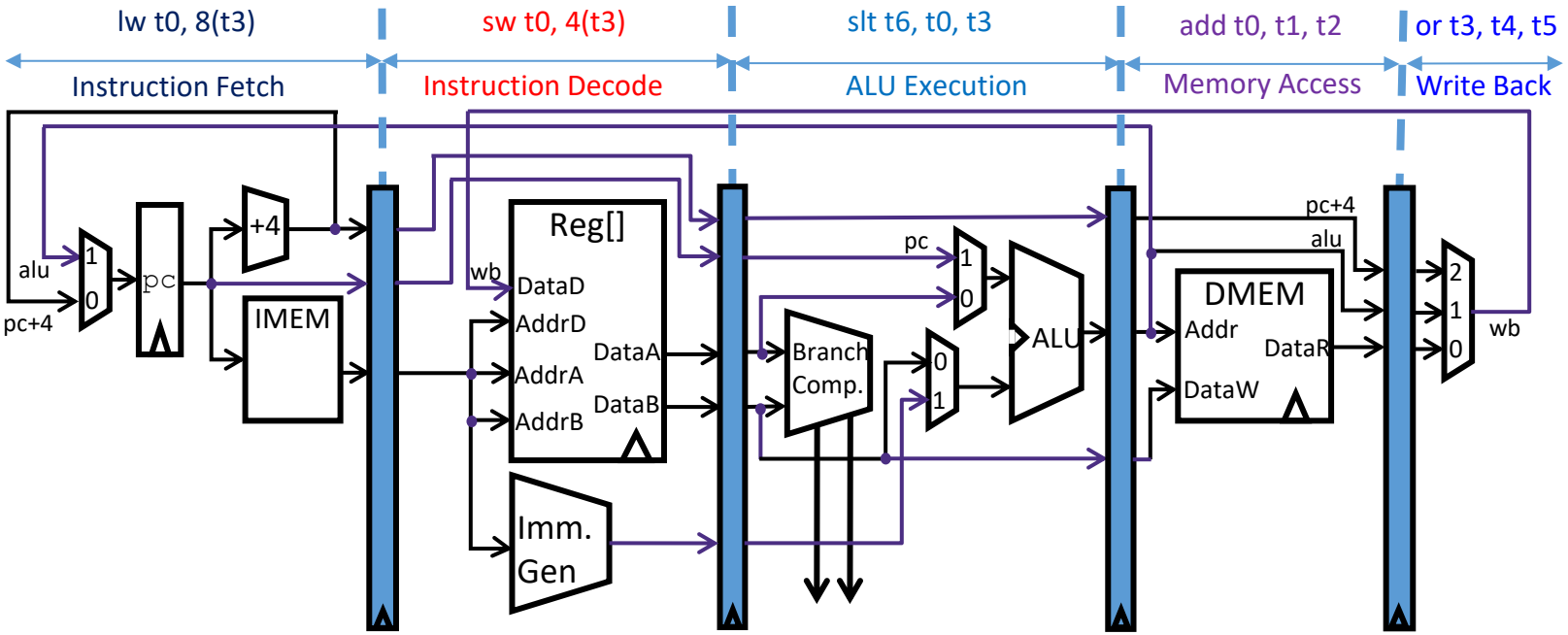
> What is the speedup when executing 1M instructions?

# Overview of a Pipelined Datapath



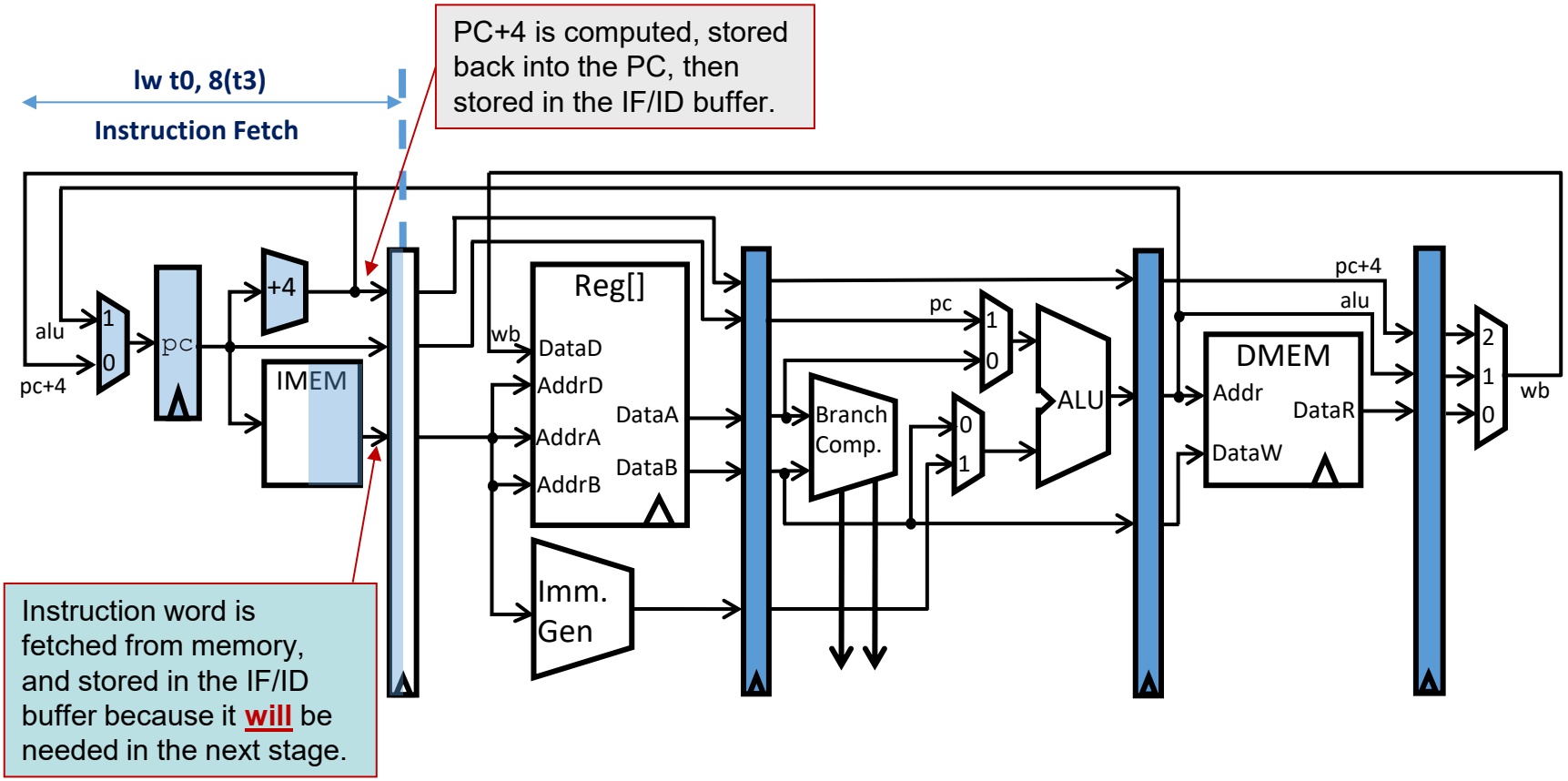

- Linear sequence of stages, each handling a distinct instruction
  - On any given cycle up to 5 instructions will be in various points of execution
  - How can we operate the stages ***independently***?

# Pipeline Registers

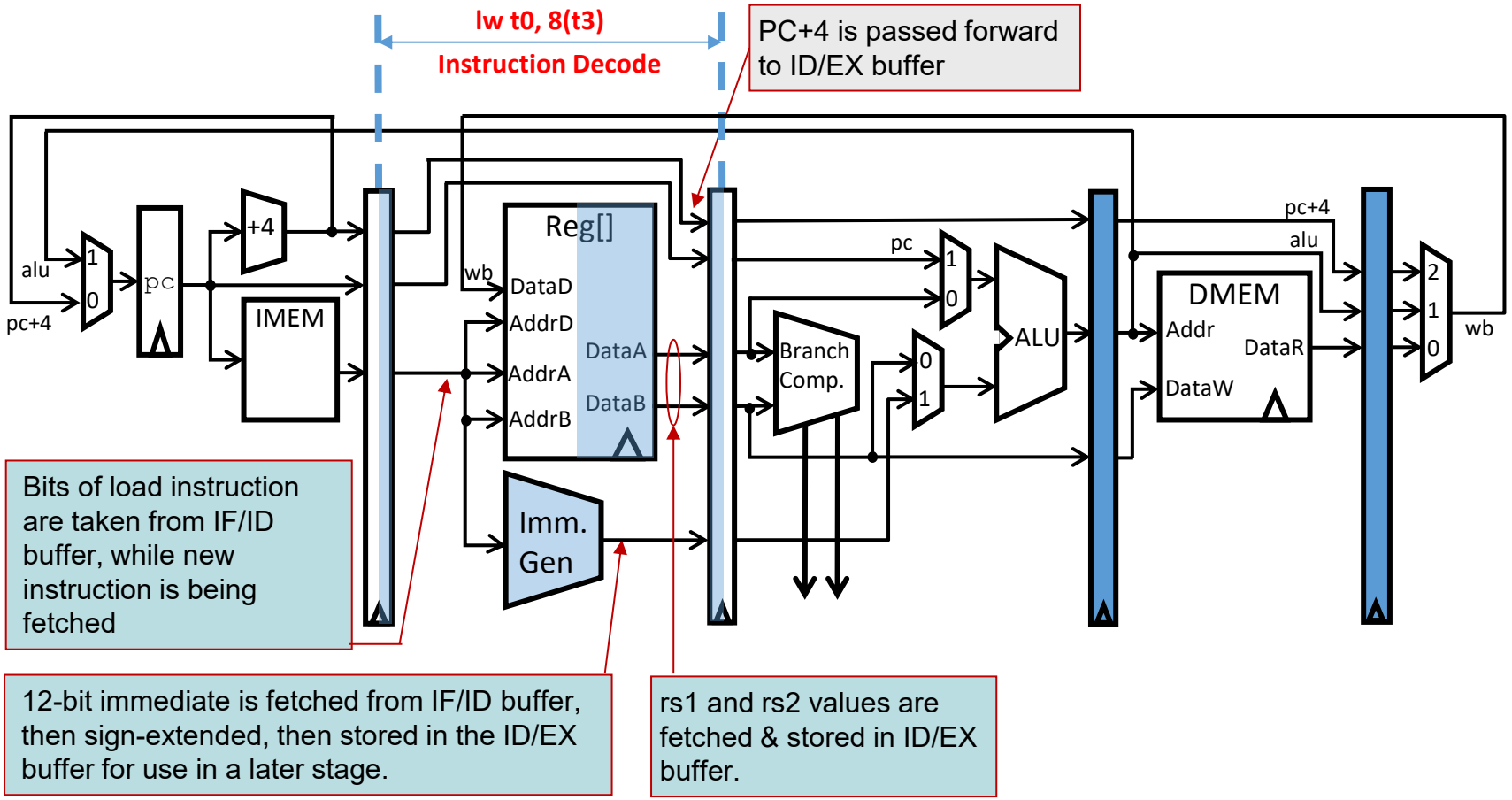



- Add state registers between each pipeline stage
  - To ***isolate*** information between cycles & hold data for each instruction in flight
  - Now, let's check the flow of instructions through the pipeline cycle-by-cycle!

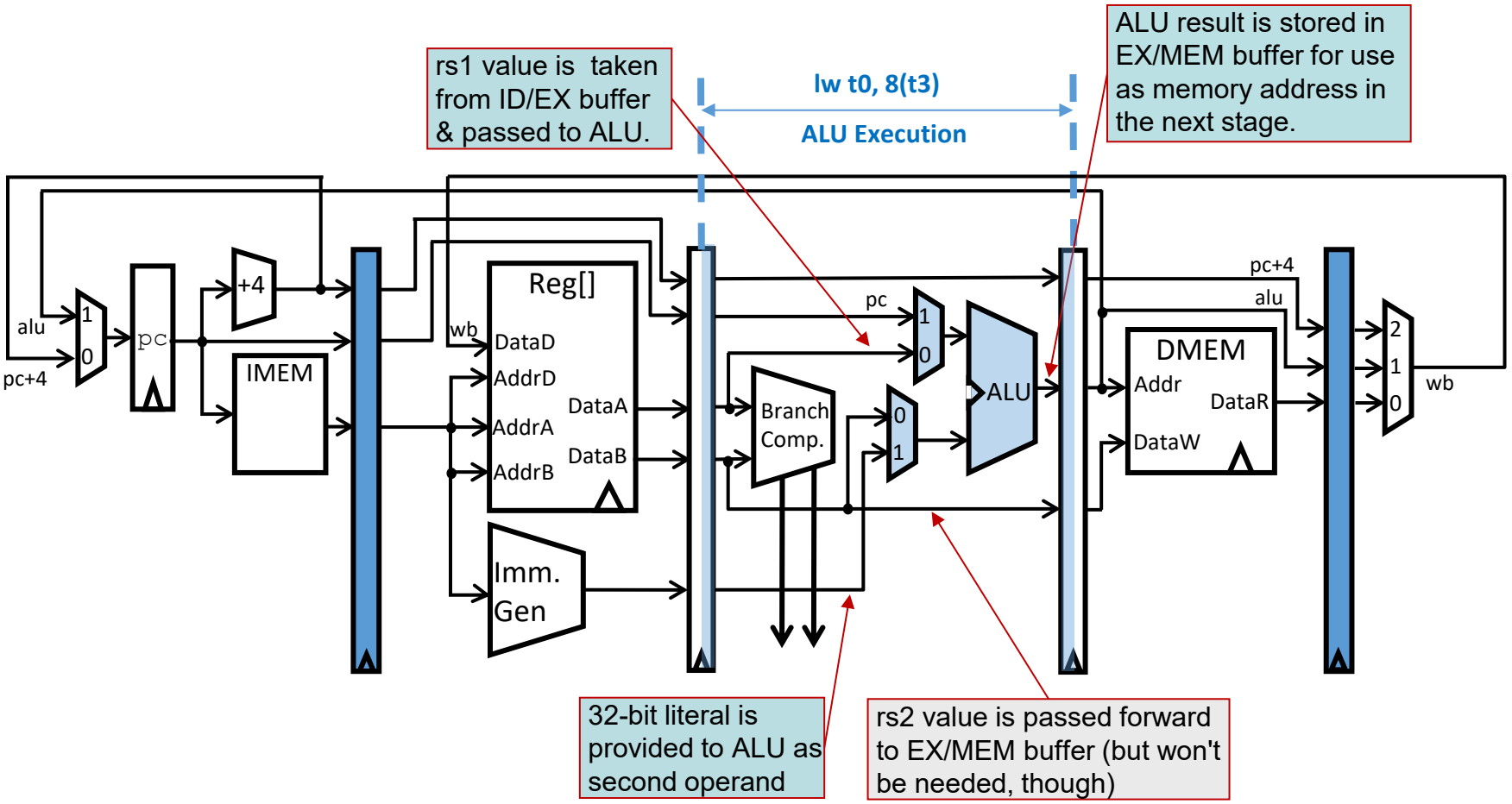
# Pipelined processor: IF for Load



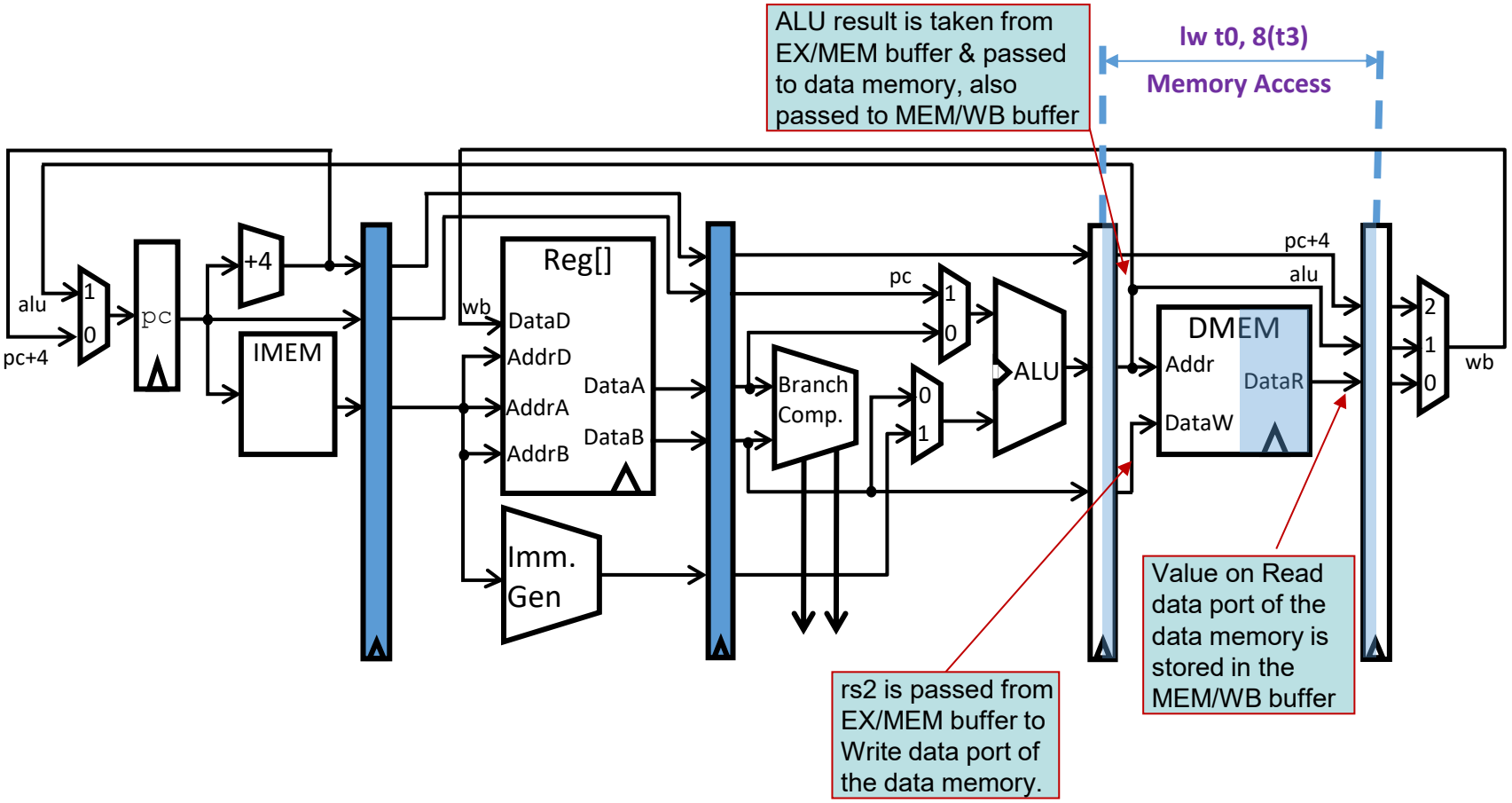
# Pipelined processor: ID for Load



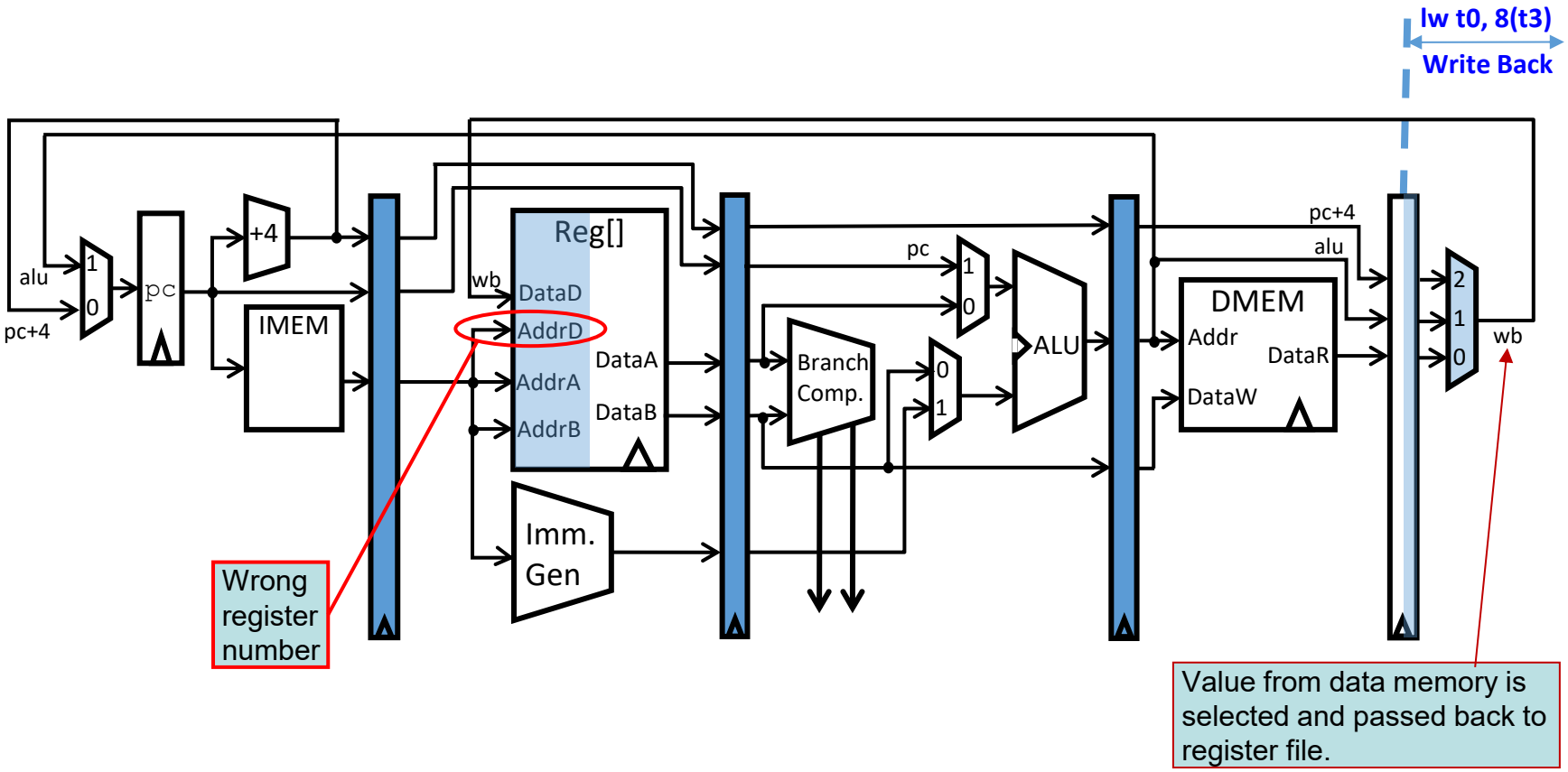
# Pipelined processor: EX for Load



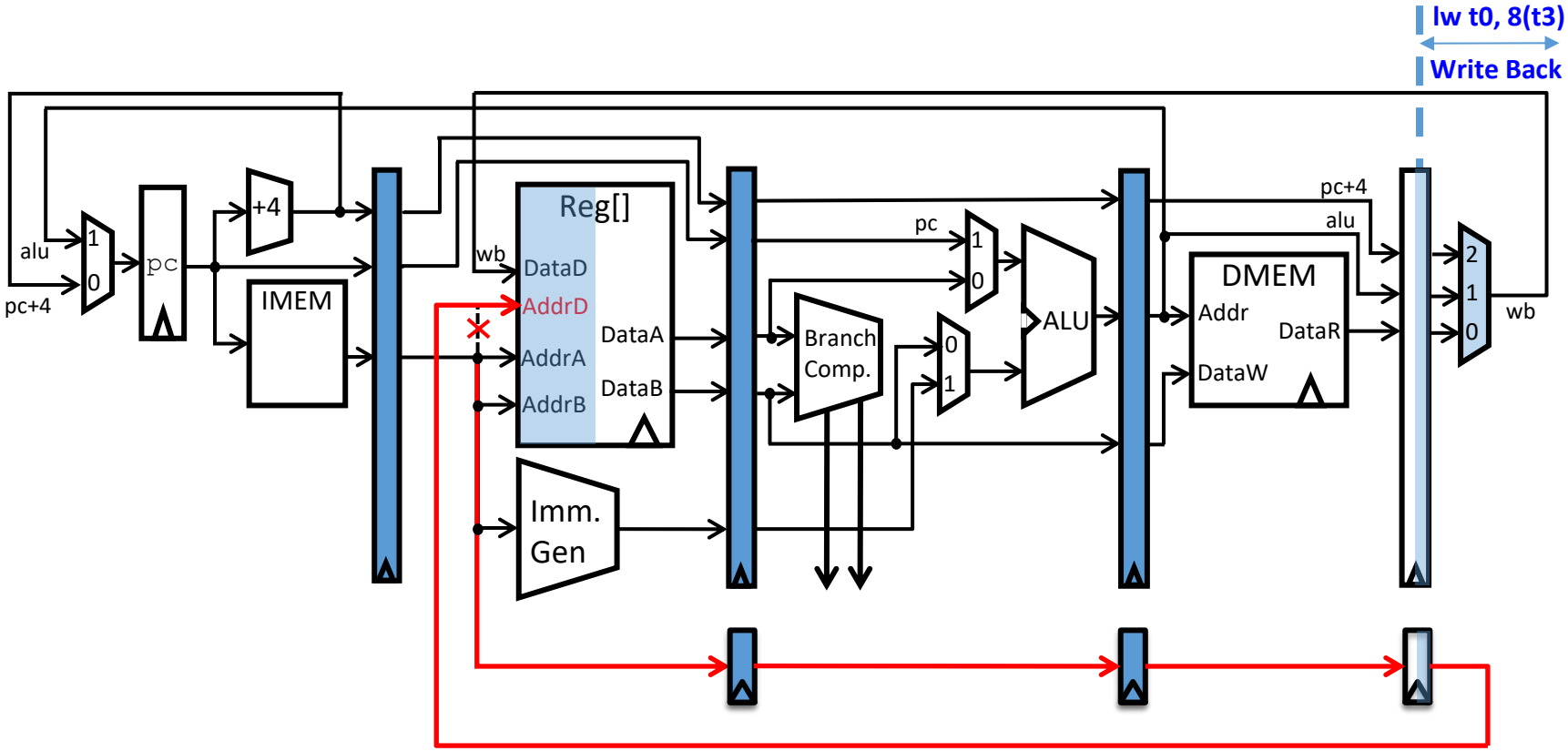
# Pipelined processor: MEM for Load



# Pipelined processor: WB for Load



# Pipelined processor: Corrected Datapath

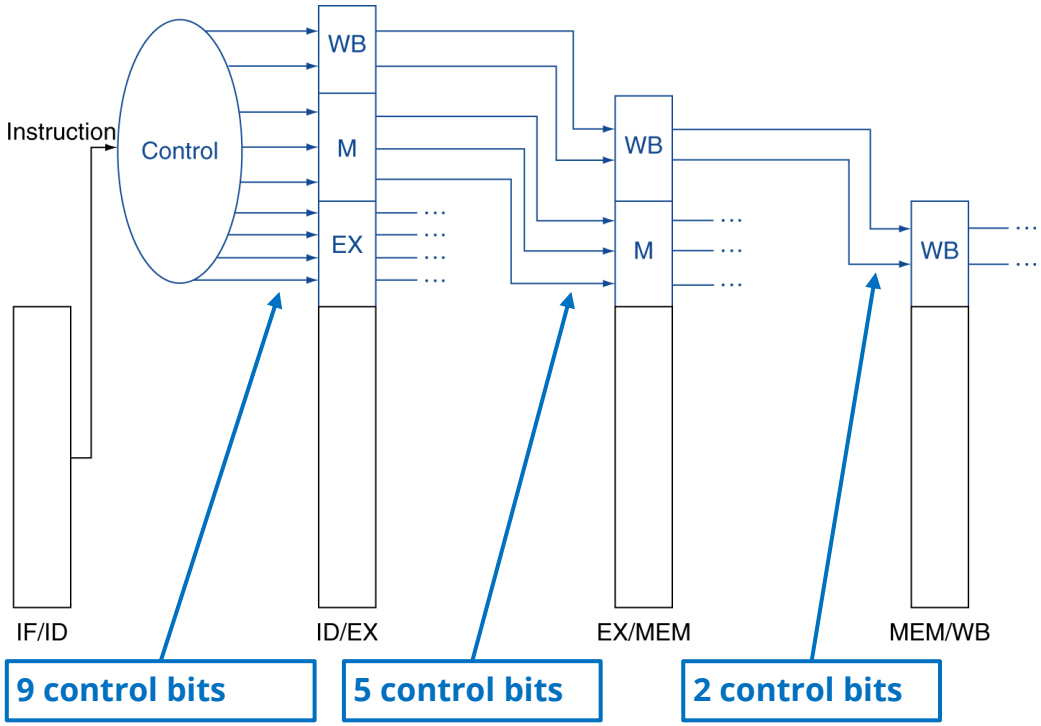



The problem is fixed by passing the Write register number through the various inter-stage buffers & feed it back just in time → adding 5 more bits to the last three buffers.

# Pipelined processor: Control Signals

> Similar to those in single-cycle implementation

- As the instruction moves → **pipeline** the control signals
- Each stage uses some of the control signals



# Pipelining the RV32I ISA

> What make it easy

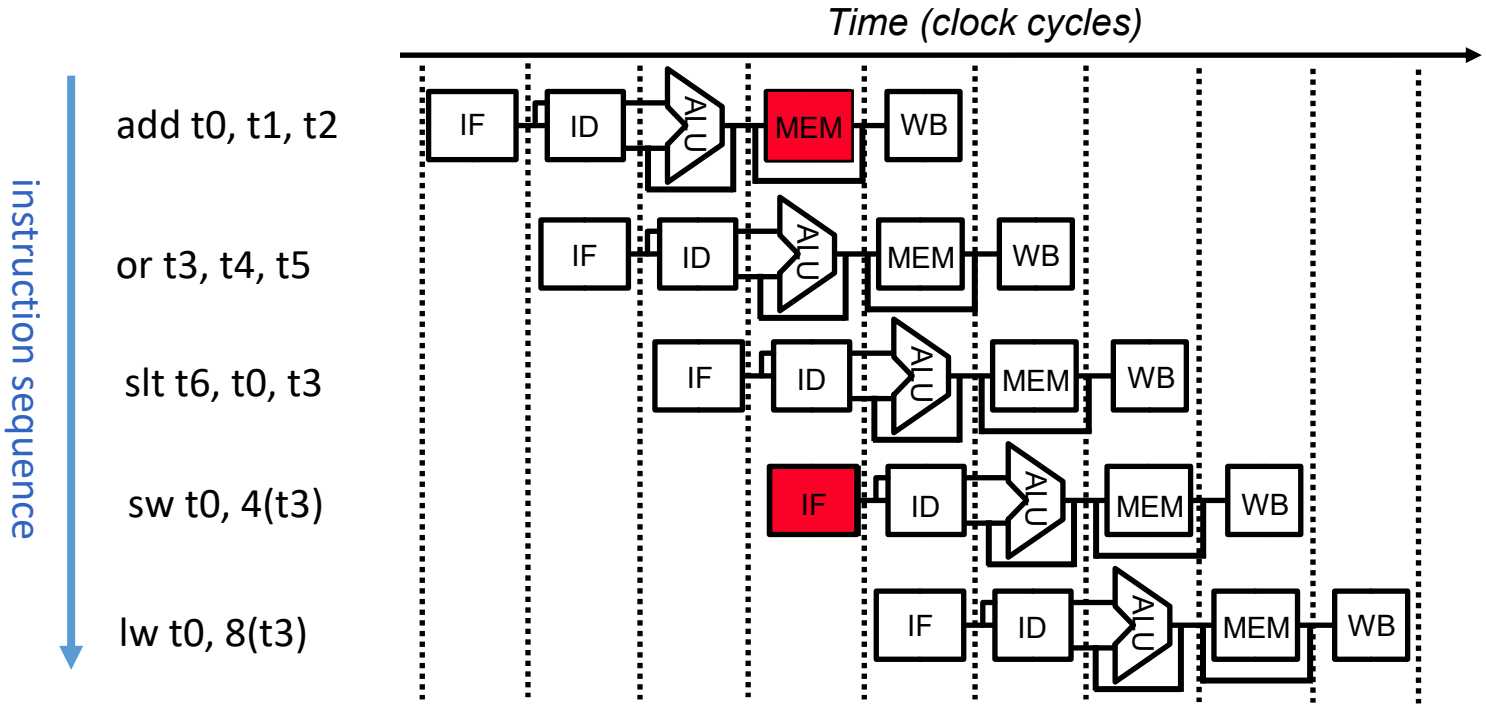
- All instructions are 32-bits: easier to fetch and decode in one cycle
  - E.g., fetch in the 1st stage and decode in the 2nd stage
- Few and regular instruction formats: can decode & read registers in 1 step
- Load/store addressing: can calculate address in the 3rd stage, and access memory in the 4th stage

> What make it hard

- Pipelining can create **hazards**: situations in which a planned instruction cannot execute in the "proper" clock cycle
  - Structural hazard
  - Data hazard
  - Control hazard
- Pipeline hazards are serious problems that cannot be ignored.

# Structural Hazard: Memory Access

> Arises when two or more instructions attempt to simultaneously access a HW resource that does not support concurrent access

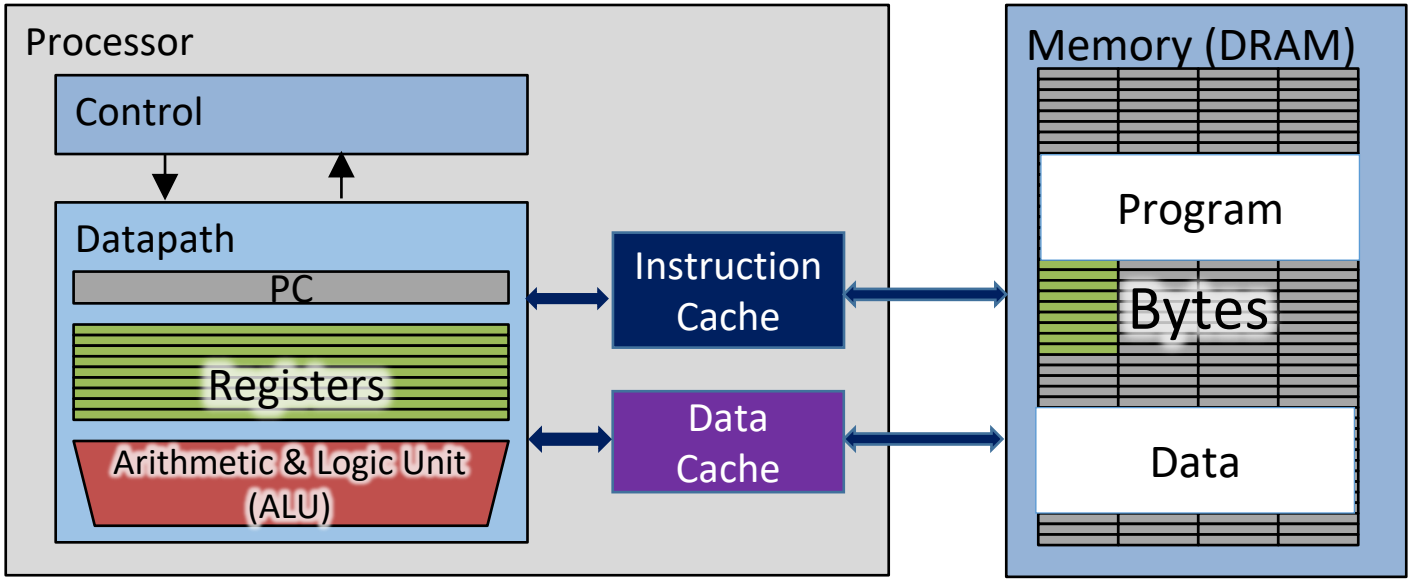



- If the main memory does **not** support concurrent read/write → hazard.

# Memory Access Hazard: Solution

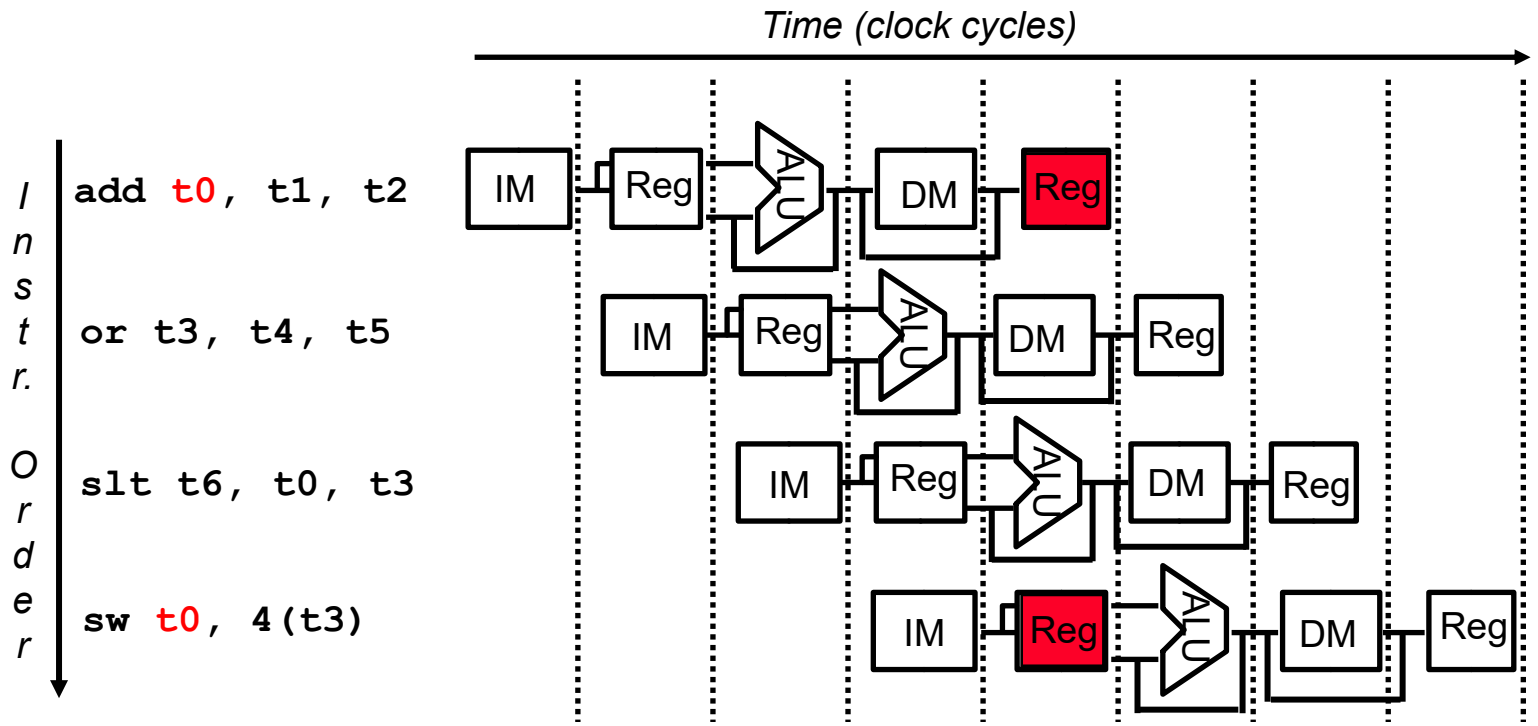
> ISA can be designed to minimize memory access hazards

- E.g., limits memory access to at most one per instruction
- Use separate caches (fast on-chip memories) for instruction & data



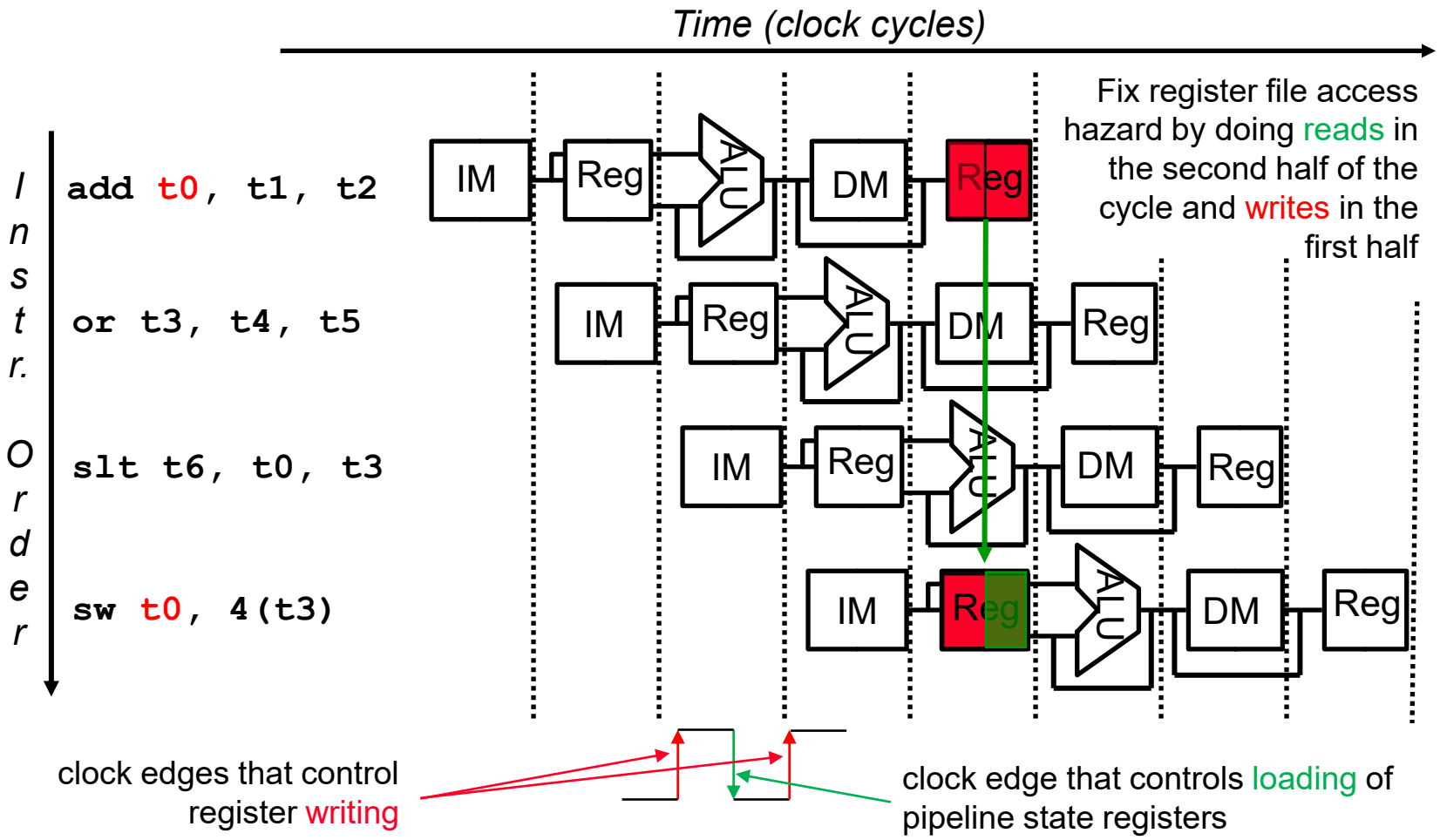
# Data Hazard: Register Access

> Occurs when an instruction tries to use data before it is ready



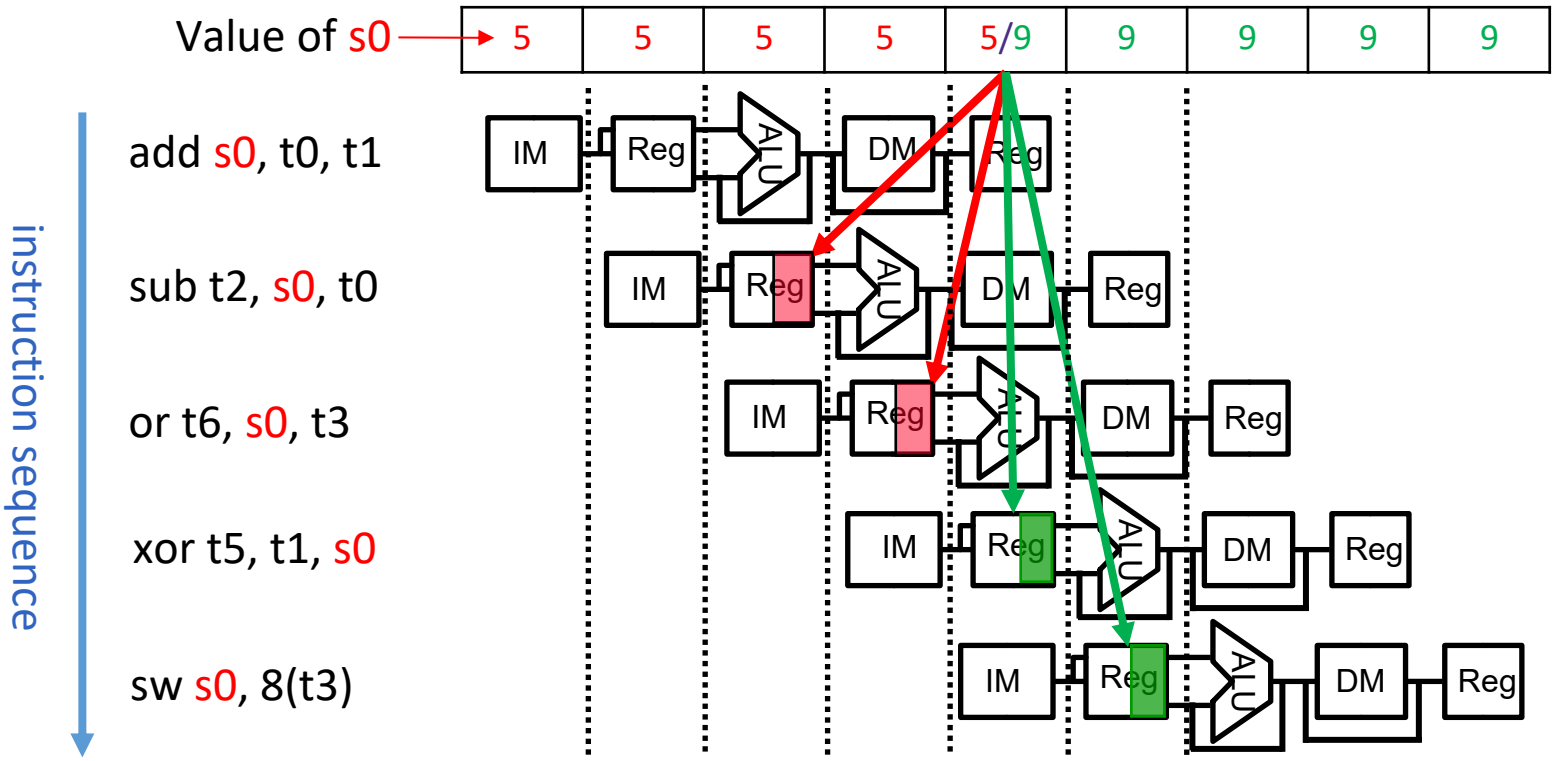

- Does **`sw`** read same value as **`add`** writes to `t0` during CC5?

# Register Access Hazard: Solution



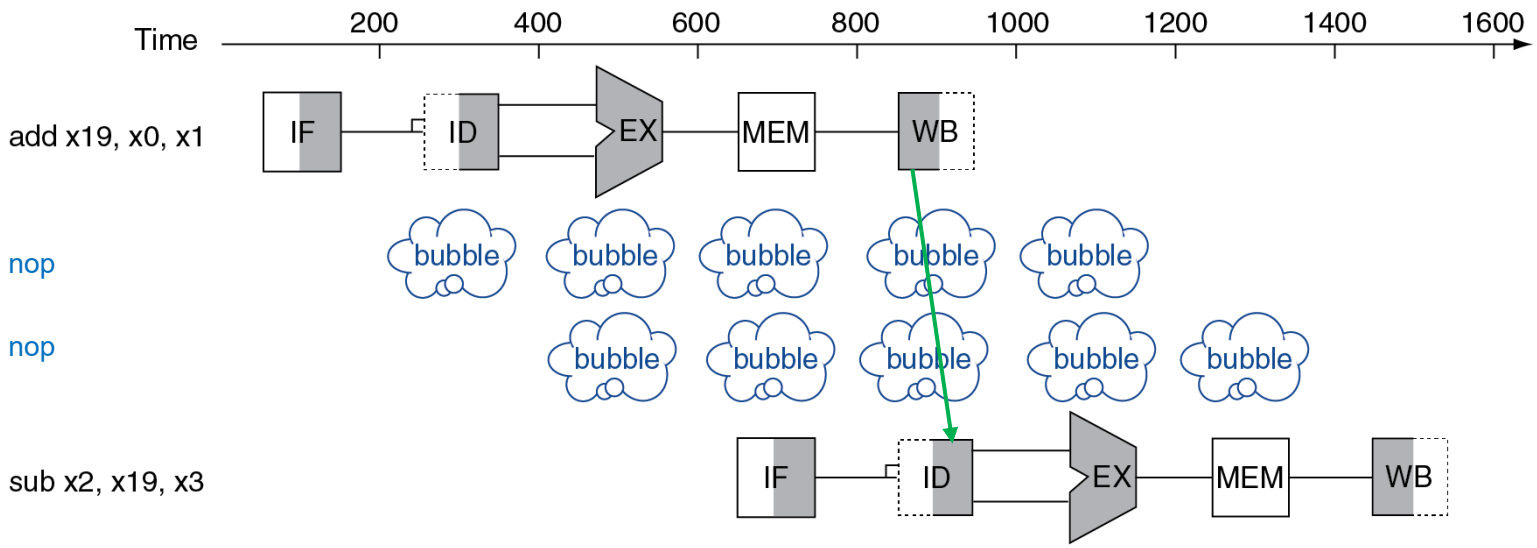
# Data Hazard: ALU Result

> Can you spot the data hazard in the following situation?




***Without some fix, `sub` and `or` will calculate wrong result!***

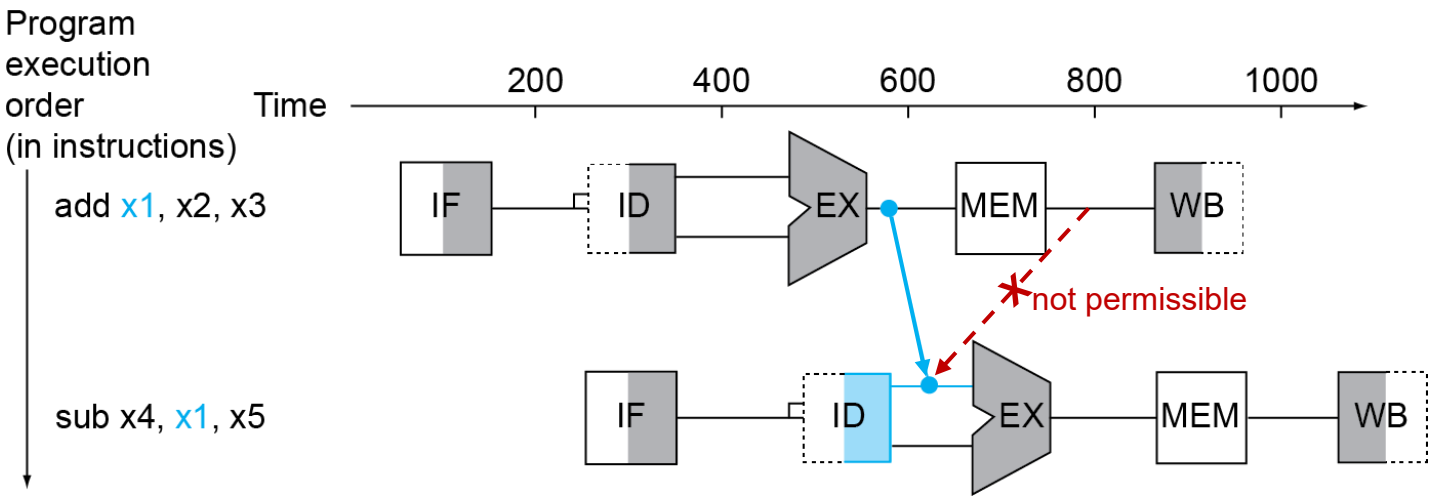
# ALU Hazard Solution: Stalling




> Pipeline stall = inserting a **`nop`** instruction into the code

- nop = no operation, often shown as bubbles in the clock diagram
- Impacts CPI, but required to get correct results

# ALU Hazard Solution: Forwarding



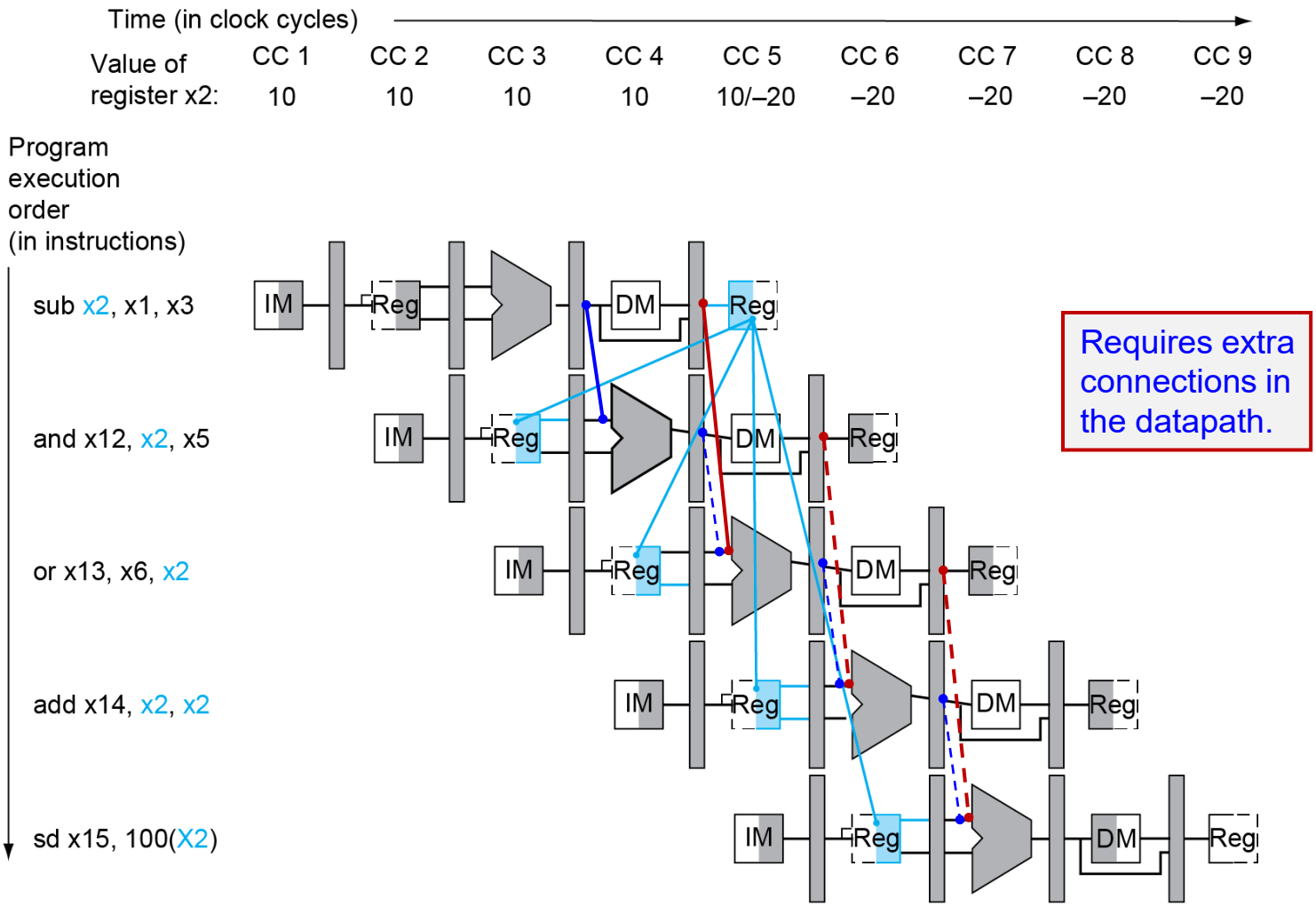

> Forward the result from the earliest state register to the ALU

- Bypassing write back → minimize latency
- Permissible only when the destination stage occurs after the source stage

> Forwarding is harder if

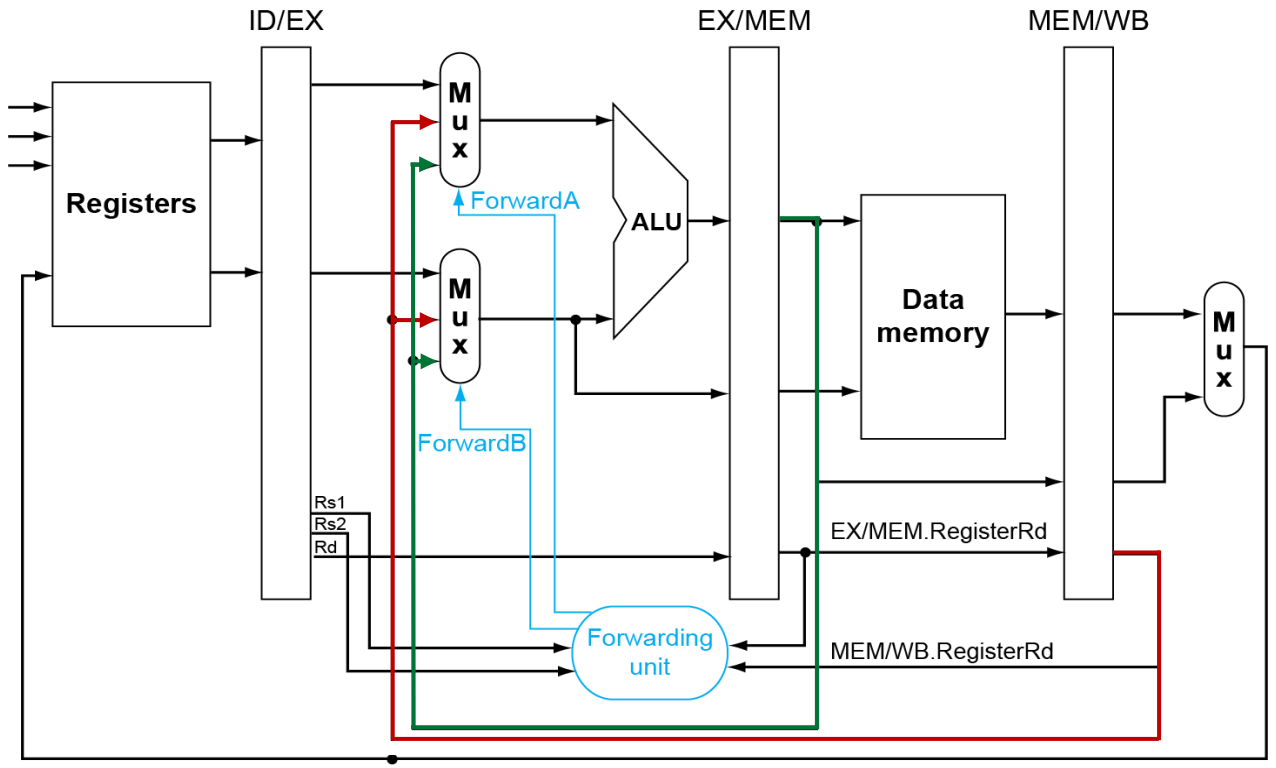
- The result is needed early in the pipeline, or
- there are multiple results to forward per instruction

# Data Forwarding: Example



# Data Forwarding: Implementation

> Data from the EX/MEM, MEM/WB pipeline buffers is fed back to two multiplexers at the inputs of the ID/EX stage




- Add a Forwarding Unit to calculate 2 control signals ForwardA & ForwardB.

# Forwarding Conditions

> Notations: Register numbers are passed along the pipeline

- E.g., EX/MEM.RegisterRd denotes the Rd register number in the EX/MEM buffer
- E.g., ALU operands in EX stage are: ID/EX.RegisterRs1, ID/EX.RegisterRs2

> Notations: Register numbers are passed along the pipeline

1. EX/MEM.RegisterRd = ID/EX.RegisterRs1
2. EX/MEM.RegisterRd = ID/EX.RegisterRs2

} Fwd from EX/MEM pipeline register

3. MEM/WB.RegisterRd = ID/EX.RegisterRs1
4. MEM/WB.RegisterRd = ID/EX.RegisterRs2

} Fwd from MEM/WB pipeline register

> … but only if forwarding instruction will write to a register

- Check if EX/MEM.RegWrite=1 (e.g. `add`), MEM/WB.RegWrite=1 (e.g. `lw`)

> … and only if `Rd` for that instruction is not `x0`

- Check if EX/MEM.RegisterRd ≠ 0, MEM/WB.RegisterRd ≠ 0

# Forwarding: Control Algorithm

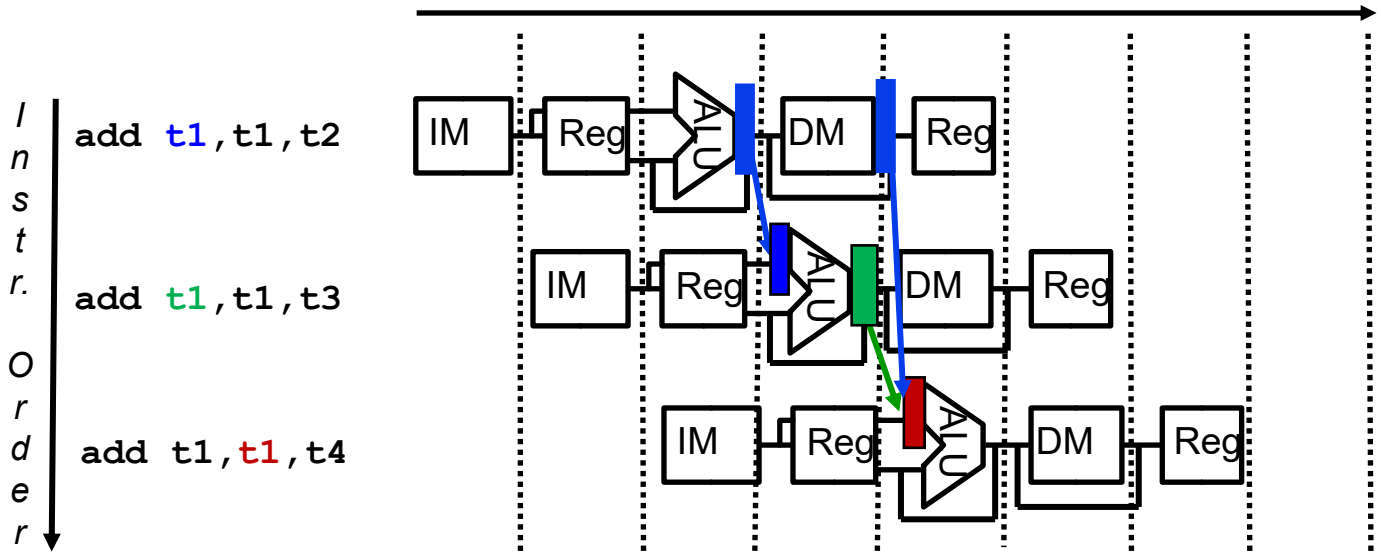
> ForwardA:

```
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0) and
(EX/MEM.RegisterRd = ID/EX.RegisterRs1)) {//EX hazard
      ForwardA = 10;
} else if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0) and
(MEM/WB.RegisterRd = ID/EX.RegisterRs1)) {//MEM hazard
      ForwardA = 01;
} else {ForwardA = 00;} //No forwarding
```

> ForwardB:

```
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0) and
(EX/MEM.RegisterRd = ID/EX.RegisterRs2)) {//EX hazard
      ForwardB = 10;
} else if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0) and
(MEM/WB.RegisterRd = ID/EX.RegisterRs2)) {//MEM hazard
      ForwardB = 01;
} else {ForwardB = 00;} //No forwarding
```

# Forwarding: Yet Another Complication



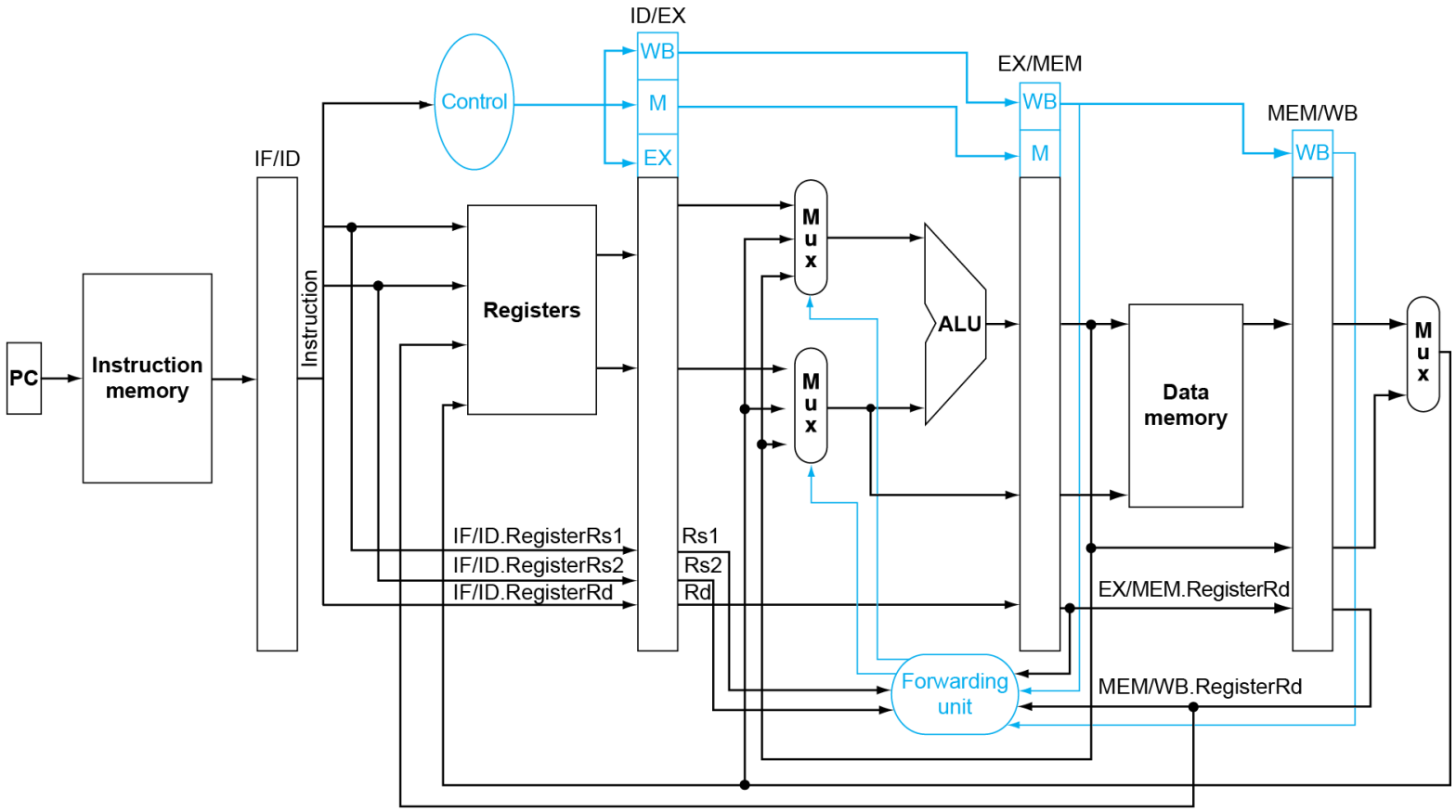

> Double data hazard:

- Both the EX & MEM stage instructions produce results that could be forwarded
- Which result should be forwarded? The more recent result (EX stage)!

> Revised FWD rule:

- Only forward MEM state if EX hazard condition isn't true

# Summary: Pipelined Datapath with Forwarding




> Does forwarding solve all our problems?

# NEXT LECTURE

[Flipped class] Pipelined processor (cont.)

- Pre-class
  - Study pre-class materials on Canvas
- In class
  - Reinforcement/enrichment discussion
  - Quizzes
- Post class
  - Homework
  - Consultation (if needed)